

An Explainable Artificial Intelligence (XAI) Framework for Network Intrusion Detection Systems

1. Introduction & Research Motivation

In modern cybersecurity infrastructures, Machine Learning (ML) algorithms are widely implemented within Network Intrusion Detection Systems (NIDS) to detect malicious network traffic and anomalous behavior. While modern tree-based ensemble models achieve exceptional accuracy rates on benchmark datasets, their internal decision-making processes remain opaque. This "black-box" nature creates significant trust and security challenges for Security Operations Center (SOC) analysts, who cannot easily verify why a specific network flow was flagged as malicious.

To address this challenge, we propose an Explainable AI (XAI) framework integrated with high-performance ML classifiers. Our goal is to build a reliable intrusion detection pipeline that delivers high classification performance while generating human-understandable, instance-level explanations for every raised security alert.

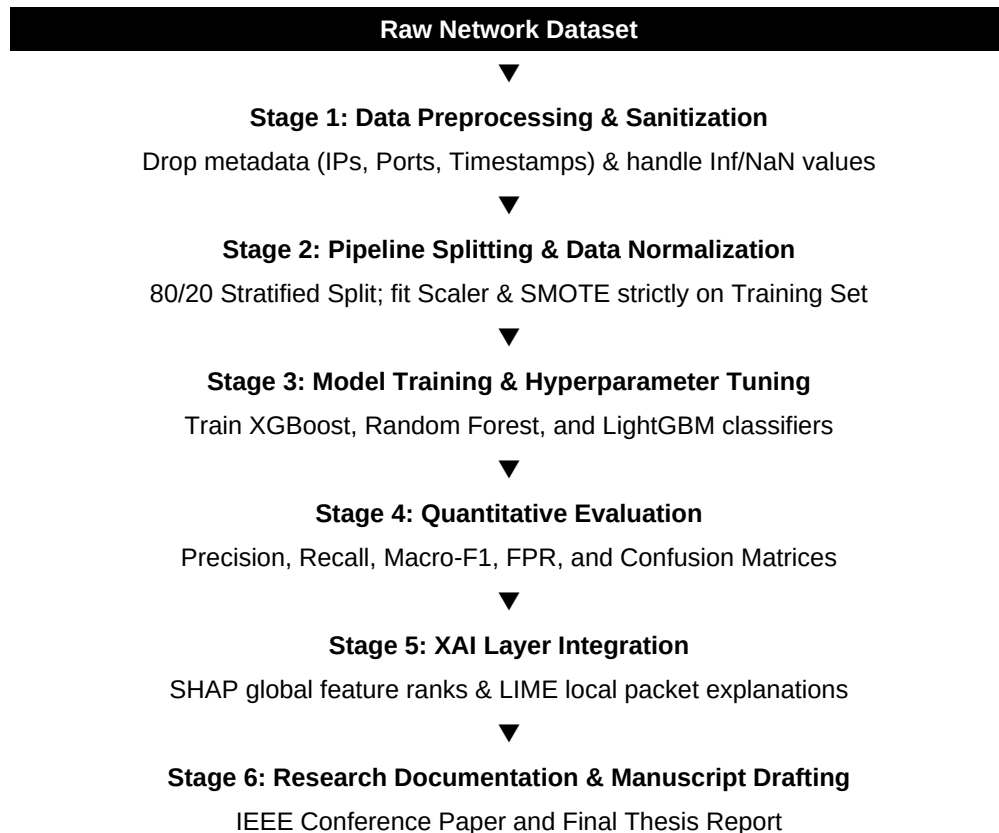
2. Project Objectives

We plan to achieve the following core objectives during this thesis project:

- 1 Build a Robust NIDS Pipeline:** We intend to preprocess benchmark network security datasets (e.g., CICIDS2017 / CICIoT2023) while strictly avoiding data spillage and target leakage.
- 2 Train State-of-the-Art Machine Learning Classifiers:** We will develop and evaluate multiple classifiers, including XGBoost, Random Forest, and LightGBM, focusing on high Macro-F1 scores and minimal False Positive Rates (FPR).
- 3 Integrate Dual-Level XAI Explanations:** We want to implement SHAP (Shapley Additive exPlanations) for global feature importance ranking and LIME (Local Interpretable Model-agnostic Explanations) for packet-level, local explanations.
- 4 Evaluate Explanation Overhead:** We plan to measure and document the inference time and explanation latency to assess the practical viability of the system in near-real-time environments.
- 5 Publish Research Findings:** We aim to author and submit a peer-reviewed research paper to an IEEE-indexed conference based on our findings.

3. Proposed Methodology & Technical Architecture

We plan to execute the project through a structured six-stage execution pipeline:



Detailed Execution Stages

- **Data Sanitization:** We will remove non-generalizable metadata features, such as Source IP, Destination IP, Source Port, Destination Port, Flow ID, and Timestamps, to prevent model overfitting and target leakage. Infinite and missing values will be handled using median imputation.
- **Pipeline Integrity:** We will execute `train_test_split` prior to any feature transformation. `MinMaxScaler` and Synthetic Minority Over-sampling Technique (SMOTE) will be fitted exclusively on the training set to prevent data leakage into the test set.
- **Explainability Layer:** We will utilize `shap.TreeExplainer` for global feature importance visualization across the entire traffic baseline, alongside `LimeTabularExplainer` to break down individual malicious packets for analyst review.

4. Team Work Division (3 Members)

To ensure efficient execution and clear accountability, we plan to divide responsibilities across three dedicated roles:

Member 1: Data Engineering & Preprocessing Lead

- Acquisition and sanitization of benchmark network datasets (CICIDS2017 / CICIoT2023).
- Execution of feature dropping, missing value imputation, and target leakage prevention.
- Implementation of feature scaling (MinMaxScaler) and class imbalance handling (SMOTE).
- Management of the team GitHub repository and data pipeline structure.

Member 2: Machine Learning & Modeling Lead

- Implementation of tree-based ensemble models (XGBoost, Random Forest, LightGBM).
- Hyperparameter tuning and optimization for each classifier.
- Evaluation of model performance metrics (Precision, Recall, Macro-F1, FPR, and Confusion Matrices).
- Comparative analysis between baseline models and optimized classifiers.

Member 3: XAI Integration & Documentation Lead

- Integration of SHAP (TreeExplainer) for global feature ranking and attribution analysis.
- Implementation of LIME for instance-level packet breakdown and local visual explanations.
- Benchmarking computation time and explanation generation latency.
- Overleaf/LaTeX setup, IEEE conference manuscript drafting, and final report compilation.

5. Project Timeline & Execution Schedule

We intend to complete this research within a **20-week (5-month)** timeframe. The project schedule is divided into five distinct four-week phases:

Execution Phase	Duration	Core Deliverables & Key Activities
Phase 1: Literature Review & Setup	Weeks 1 – 4	Literature review of 20+ peer-reviewed papers, dataset acquisition, GitHub workspace configuration, and proposal defense preparation.

Execution Phase	Duration	Core Deliverables & Key Activities
Phase 2: Preprocessing & Data Pipeline	Weeks 5 – 8	Cleaning raw CSV files, dropping metadata columns, setting up stratified split pipelines, and implementing SMOTE/scaling scripts.
Phase 3: Model Training & Evaluation	Weeks 9 – 12	Developing XGBoost, Random Forest, and LightGBM models, hyperparameter tuning, and generating evaluation metrics.
Phase 4: XAI Layer Integration	Weeks 13 – 16	Integrating SHAP and LIME, extracting global and local explanation plots, and measuring system latency.
Phase 5: Manuscript Drafting & Defense	Weeks 17 – 20	Authoring the IEEE conference paper draft on Overleaf, preparing the complete thesis book, and finalizing the slide deck for defense.

6. Expected Deliverables

Upon completion of this project, we plan to deliver the following items:

- 1 **Open-Source Codebase:** A clean, well-documented GitHub repository containing data preprocessing scripts, trained ML models, and XAI explanation modules.
- 2 **IEEE Research Paper Draft:** A complete conference manuscript formatted according to IEEE guidelines, targeting submission to an IEEE-indexed conference.
- 3 **Comprehensive Thesis Book:** A formal thesis report detailing our methodology, experiments, results, and analysis.
- 4 **Interactive Explanation Visuals:** A collection of global feature attribution plots (SHAP) and individual packet decision breakdowns (LIME) illustrating system interpretability.